Searching Algorithm

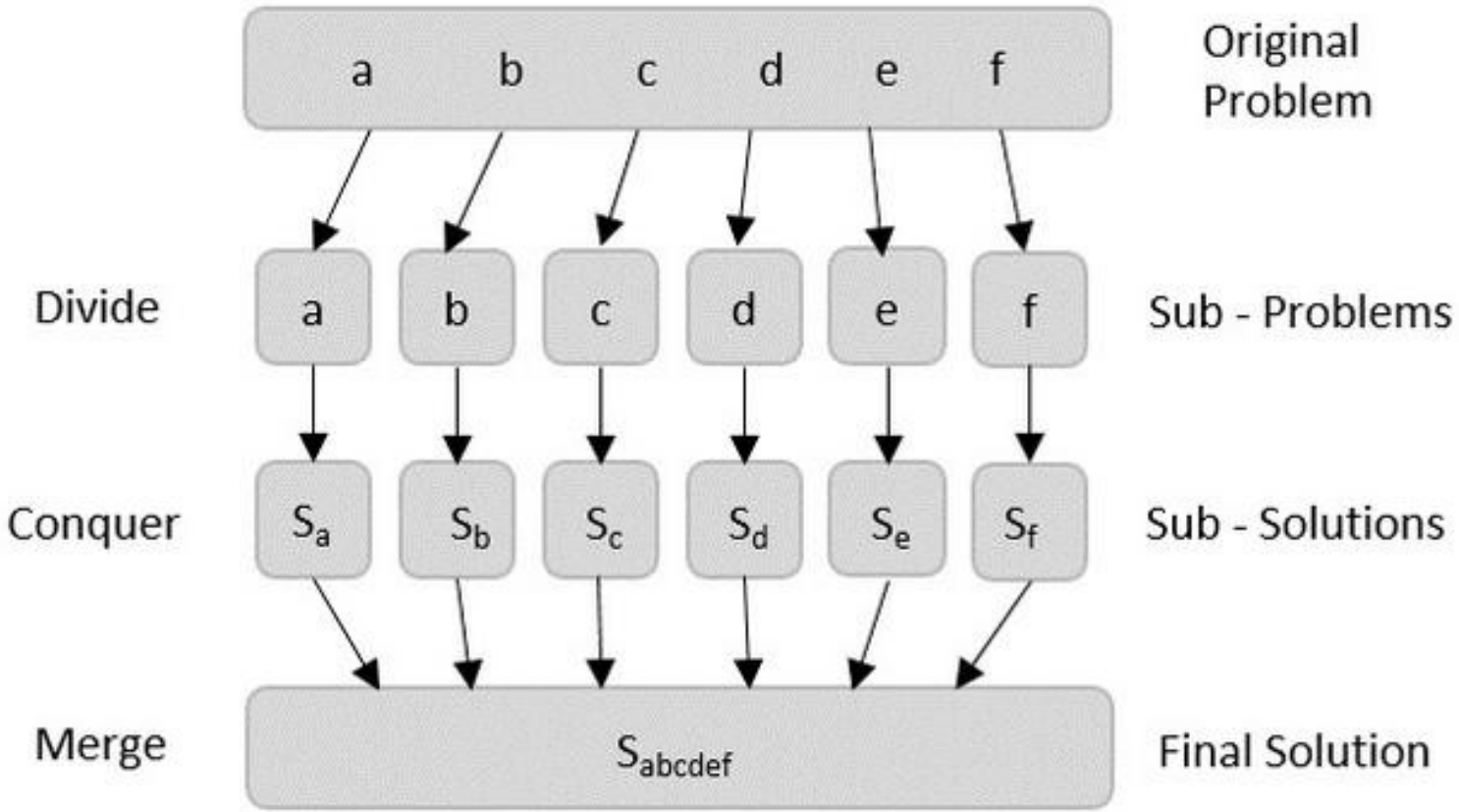
Divide and Conquer

- In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently. When we keep on dividing the subproblems into smaller sub-problems, we may eventually reach a stage where no more division is possible. The solution of all sub-problems is finally merged in order to obtain the solution of an original problem.

Cont..

- **Divide:** This step involves breaking the problem into smaller sub-problems. Sub-problems should represent a part of the original problem. This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible. At this stage, sub-problems become atomic in nature but still represent some part of the actual problem.
- **Conquer:** This step receives a lot of smaller sub-problems to be solved. Generally, at this level, the problems are considered 'solved' on their own.
- **Merge**: When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem. This algorithmic approach works recursively and conquer & merge steps works so close that they appear as one.

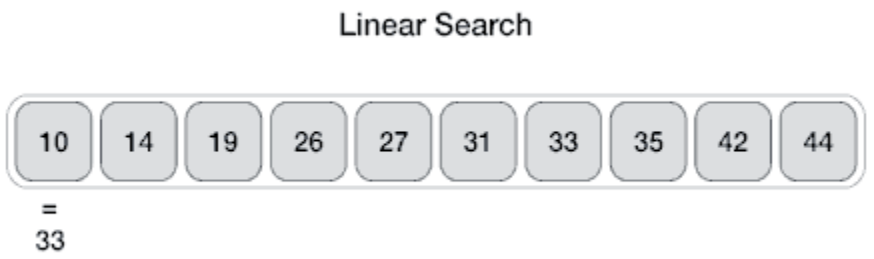
Divide & Conquer



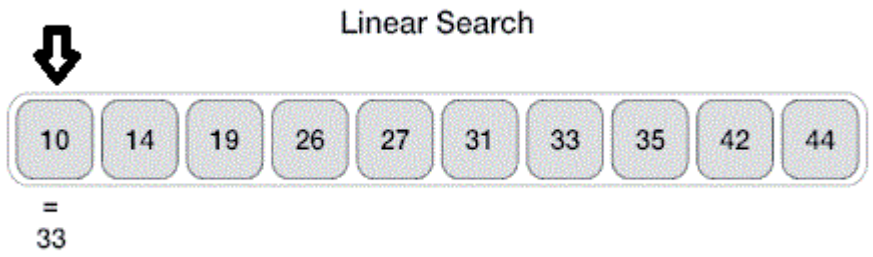
Linear Search

- Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection

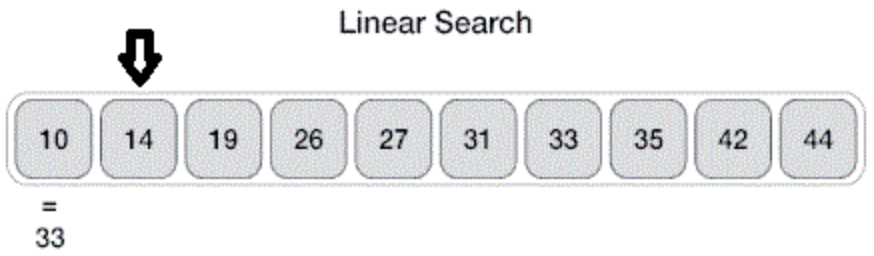
Linear Search



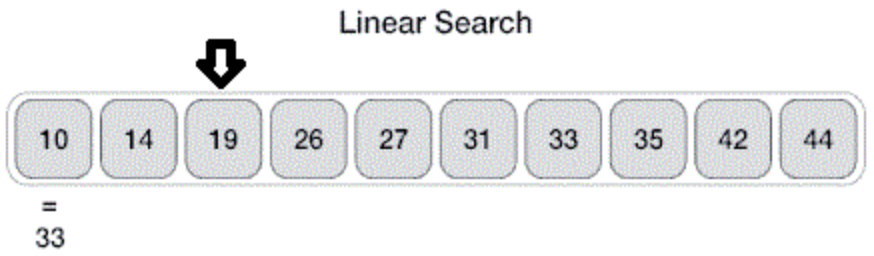
Linear Search



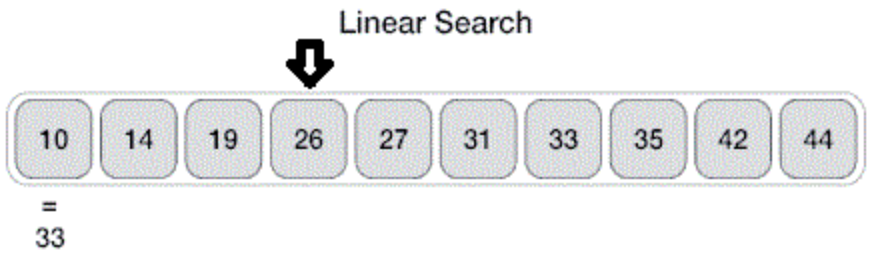
Linear Search



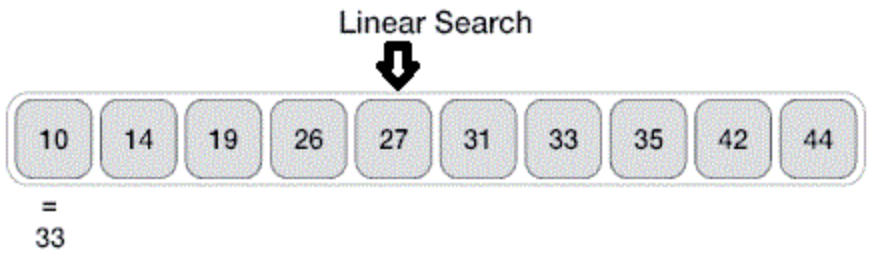
Linear Search



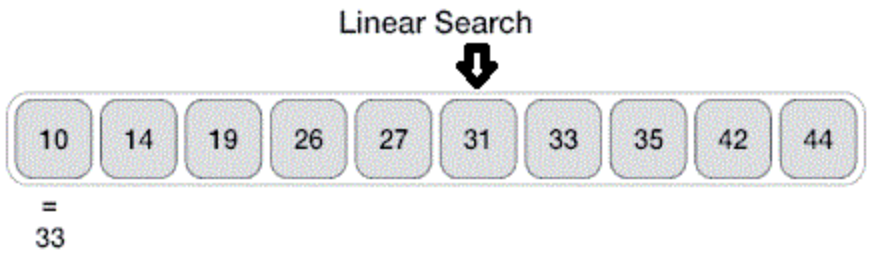
Linear Search



Linear Search



Linear Search



Linear Search



Linear Search Steps

- Start from the leftmost element of arr[] and one by one compare x with each element of arr[]
- If x matches with an element, return the index.
- If x doesn’t match with any of elements, return -1.

Linear Search (Sample)

- **What is Complexity of this Algorithm???**

```
int search(int arr[], int n, int x)
{  int i;
        for (i = 0; i < n; i++)
                if (arr[i] == x)
                        return i;
}
int main(void)
{
        int arr[] = { 2, 3, 4, 10, 40 };
        int x = 10;
        int n = sizeof(arr) / sizeof(arr[0]);
        int result = search(arr, n, x);
        (result == -1)? cout<<"Element is not present in array“ : cout<<"Element is present at index " <<result;
return 0;
}
```

Linear Search

```
procedure linear_search (list, value)
for each item in the list
        if match item == value
                return the item's location
        end if
end for
end procedure
```

Binary Search

- **Binary search** is the most popular Search algorithm. It is efficient and also one of the most commonly used techniques that is used to solve problems.
- Binary search works only on a sorted set of elements. To use binary search on a collection, the collection must first be sorted.
- When binary search is used to perform operations on a sorted set, the number of iterations can always be reduced on the basis of the value that is being searched.

Binary Search

- This search algorithm works on the principle of divide and conquer. For this algorithm to work properly, the data collection should be in the sorted form.
- Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub-array as well until the size of the subarray reduces to zero

Cont..

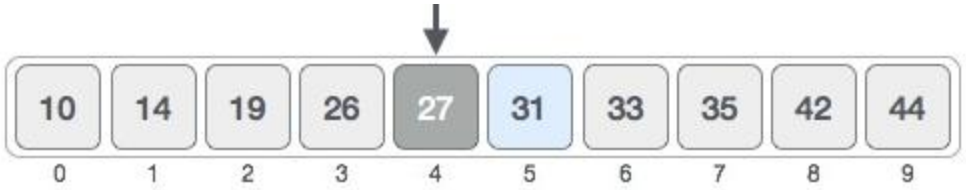
- The following is our sorted array and let us assume that we need to search the location of value 31 using binary search



Binary Search

- First, we shall determine half of the array by using this formula −

$$mid = low + (high - low) / 2$$



Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

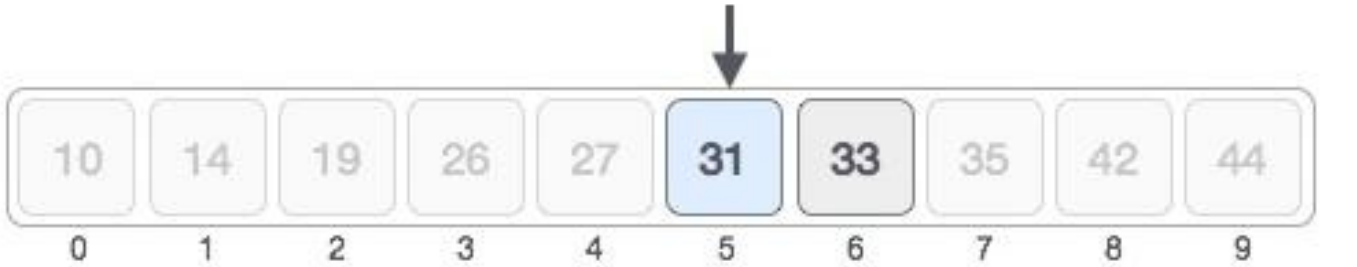
Binary Search



Binary Search



Binary Search





- We conclude that the target value 31 is stored at location 5.

Implementing Binary Search

Following are the steps of implementation that we will be following:

1. Start with the middle element:
    - If the **target** value is equal to the middle element of the array, then return the index of the middle element.
    - If not, then compare the middle element with the target value,
        - If the target value is greater than the number in the middle index, then pick the elements to the right of the middle index, and start with Step 1.
        - If the target value is less than the number in the middle index, then pick the elements to the left of the middle index, and start with Step 1.
2. When a match is found, return the index of the element matched.
3. If no match is found, then return -1

Binary Search (Sample)

- **What is Complexity of this algorithm ??**

```
int binarySearch(int arr[], int l, int r, int x)
{
        while (l <= r) {
                int m = l + (r - l) / 2;
                // Check if x is present at mid
                if (arr[m] == x)
                        return m;
                // If x greater, ignore left half
                if (arr[m] < x)
                        l = m + 1;
                // If x is smaller, ignore right half
                else
                        r = m - 1;
        }
        // if we reach here, then element was
        // not present
        return -1;
```

Difference

Linear Search

- A linear search scans one item at a time, without jumping to any item .
- The worst case complexity is O(n), sometimes known an O(n) search
- Time taken to search elements keep increasing as the number of elements are increased.

Binary Search

- A binary search however, cut down your search to half as soon as you find middle of a sorted list.
- The middle element is looked to check if it is greater than or less than the value to be searched.
- Accordingly, search is done to either half of the given list

Algorithms

- In an algorithm design there is no one 'silver bullet' that is a cure for all computation problems. Different problems require the use of different kinds of techniques. A good programmer uses all these techniques based on the type of problem. Some commonly-used techniques are:
- Divide and conquer
- Randomized algorithms
- Greedy algorithms (This is not an algorithm, it is a **technique**.)
- Dynamic programming

Greedy Algorithm

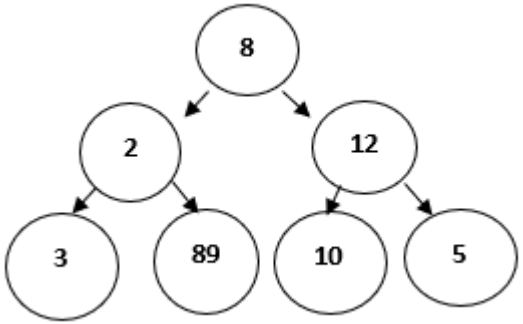
- A greedy algorithm, as the name suggests, **always makes the choice that seems to be the best at that moment**. This means that it makes a locally-optimal choice in the hope that this choice will lead to a globally-optimal solution.

- Consider that there is an **objective function** that has to be optimized (maximized/ minimized). This approach makes greedy choices at each step and makes sure that the objective function is optimized.

- The greedy algorithm has only one chance to compute the optimal solution and thus, cannot go back and look at other alternate solutions. However, in many problems, this strategy fails to produce a global optimal solution.

Greedy Algorithm

- Let's consider the following binary tree to understand how a basic greedy algorithm works:



For the above problem the objective function is:

- **To find the path with largest sum.**

Since we need to maximize the objective function, Greedy approach can be used. Following steps are followed to find the solution:

Greedy Algorithm

Step 1: Initialize **sum = 0**
Step 2: Select the root node, so its value will be added to sum, **sum = 0+8 = 8**
Step 3: The algorithm compares nodes at next level, selects the largest node which is **12**, making the **sum = 20**.
Step 4: The algorithm compares nodes at the next level, selects the largest node which is **10**, making the **sum = 30**.

- Thus, using the greedy algorithm, we get **8-12-10** as the path. But this is not the optimal solution, since the path **8-2-89** has the largest sum i.e. **99**.
- This happens because the algorithm makes decision based on the information available at each step without considering the overall problem.

When to use Greedy Algorithms?

For a problem with the following properties, we can use the greedy technique:

- **Greedy Choice Property**: This states that a globally optimal solution can be obtained by locally optimal choices.
- **Optimal Sub-Problem**: This property states that an optimal solution to a problem, contains within it, optimal solution to the sub-problems. Thus, a globally optimal solution can be constructed from locally optimal sub-solutions.

- Generally, **optimization problem**, or the problem where we have to find maximum or minimum of something or we have to find some optimal solution, greedy technique is used.

Application

Most networking algorithms use the greedy approach. Here is a list of few of them −

- Travelling Salesman Problem
- Prim's Minimal Spanning Tree Algorithm
- Kruskal's Minimal Spanning Tree Algorithm
- Dijkstra's Minimal Spanning Tree Algorithm
- Graph - Map Coloring
- Graph - Vertex Cover
- Knapsack Problem
- Job Scheduling Problem

Dynamic Programming

- Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems.
- But unlike, divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.

Dynamic Programming

- Dynamic programming is used where we have problems, which can be divided into similar sub-problems, so that their results can be re-used. Mostly, these algorithms are used for optimization. Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems. The solutions of sub-problems are combined in order to achieve the best solution

Comparison

- In contrast to greedy algorithms, where local optimization is addressed, dynamic algorithms are motivated for an overall optimization of the problem.
- In contrast to divide and conquer algorithms, where solutions are combined to achieve an overall solution, dynamic algorithms use the output of a smaller sub-problem and then try to optimize a bigger sub-problem. Dynamic algorithms use Memoization to remember the output of already solved sub-problems.